Snake-OS: A Bare-Metal, High-Fidelity Terminal Gaming Experience

Pintu Singh
Roll No: 230105
B.Tech CS & AI
Newton School of Technology
Sonipat, India

Pranay Sarkar
Roll No: 230047
B.Tech CS & AI
Newton School of Technology
Sonipat, India

Fathal
Roll No: 230043
B.Tech CS & AI
Newton School of Technology
Sonipat, India

***Abstract*—Snake-OS is a fully functional Snake game implemented in the C programming language that deliberately avoids standard library facilities to simulate a bare-metal, operating-system-level programming environment. The project replaces standard functions such as `malloc()`, `free()`, `rand()`, and `sprintf()` with custom implementations, and further prohibits the use of the hardware multiplication, division, and modulo operators. All rendering is performed using ANSI/VT100 escape sequences written directly to standard output, eliminating any dependency on libraries such as *ncurses*. Terminal input is managed through the POSIX `termios` API to achieve non-blocking, raw-mode keyboard handling. The game features two distinct play modes (CLASSIC and INFINITY), a tiered food reward system, dynamic obstacle generation, animated death sequences, and a board that adapts in real time to terminal window resizes. This paper presents the system architecture, module-level design decisions, and algorithmic implementations.**

***Index Terms*—C programming, custom memory allocator, terminal I/O, ANSI escape codes, linked list, bare-metal, Snake game, termios, low-level systems**

I. Introduction

Modern application development heavily relies on high-level abstractions provided by operating systems and runtime libraries. While these abstractions accelerate development, they conceal the underlying mechanisms that govern memory management, arithmetic operations, and input/output processing. Snake-OS is designed as an educational systems programming project that deliberately strips away these abstractions, requiring the developer to implement each layer from the ground up.

The project draws its name from this philosophy: like an operating system, Snake-OS manages its own memory pool, implements its own arithmetic primitives, handles its own terminal I/O, and renders its own graphics – all within the constraints of a standard C source file compiled against only the most minimal POSIX system calls.

The primary objectives of this project are:

- To implement a custom heap allocator on a fixed-size memory buffer without using `malloc()` or `free()`.
- To perform all integer arithmetic using only addition and subtraction, replacing the `*`, `/`, and `%` operators.
- To achieve real-time, non-blocking keyboard input without requiring the user to press the Enter key.
- To render a complete, animated, coloured game interface using only ANSI escape code sequences written to `stdout`.
- To produce a playable, polished game that demonstrates these low-level concepts in a tangible, interactive form.

The remainder of this paper is organised as follows. Section II describes the system architecture. Sections III through VIII detail each module. Section IX presents results, and Section XI concludes the paper.

II. System Architecture

Snake-OS is organised into six C source modules, each with a corresponding header file. The game engine (`snake.c`) acts as the top-level coordinator and depends on all five supporting modules, as shown in Figure 1.

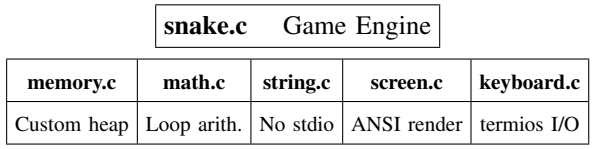

Fig. 1. Module dependency diagram of Snake-OS.

Table I summarises the responsibility and key constraint of each module.

TABLE I
Snake-OS Module Summary

Module	Responsibility	Key Constraint
memory.c	Heap alloc & dealloc	No `malloc`/`free`
math.c	Integer arithmetic	No `*`, `/`, `%`
string.c	String operations	No `stdio.h`
screen.c	Terminal rendering	No *ncurses*
keyboard.c	Raw key input	No blocking reads
snake.c	Game logic	All modules combined

The build system is a single `Makefile` that compiles all six source files with `gcc -Wall -Iinclude` and links them into a single executable named `snake`.

III. Memory Management – memory.c

A. Design Overview

The allocator operates on a globally declared static array:

```
char VRAM[8192]; /* 8 KB static memory pool */
int  g_heap_end; /* current end of used region */
```

Every allocation is preceded by a `BlockHeader` struct embedded directly in the `VRAM` array:

```
typedef struct BlockHeader {
    int size;    /* usable bytes after header */
    int is_free; /* 1 = free, 0 = allocated   */
} BlockHeader;
```

B. Alignment

Before every allocation the requested size is rounded up to the nearest multiple of 8 bytes using `align_up()`. This prevents bus errors on architectures that require aligned memory access.

C. First-Fit Allocation

`my_alloc()` scans `VRAM` from offset 0, inspecting each `BlockHeader`. The first free block whose `size` field satisfies the request is selected. If the remaining space after the allocation exceeds `sizeof(BlockHeader) + sizeof(void*)`, the block is split into an allocated portion and a smaller free block. This reduces internal waste without the overhead of a Best-Fit exhaustive search.

D. Deallocation and Coalescing

`my_dealloc()` recovers the header by subtracting `sizeof(BlockHeader)` from the user pointer, sets `is_free = 1`, and invokes `coalesce_forward()`. This function iteratively merges the freed block with any immediately following free blocks, preventing external fragmentation:

```
void coalesce_forward(BlockHeader *hdr) {
    while (1) {
        next = (BlockHeader *)((char *)hdr
               + sizeof(BlockHeader) + hdr->size);
        if ((char *)next >= &VRAM[g_heap_end]) return;
        if (!next->is_free) return;
        hdr->size += sizeof(BlockHeader) + next->size;
    }
}
```

IV. Arithmetic Primitives – math.c

A. Motivation

The project prohibits the C `*`, `/`, and `%` operators to simulate a minimal instruction-set environment where hardware multiply/divide units are unavailable – a constraint common in early 8-bit microcontrollers.

B. Implementation

All four arithmetic primitives are implemented using repeated addition or subtraction. Table II summarises their algorithms and time complexities.

Sign handling is performed by tracking a sign variable before operating on absolute values, correctly implementing two's-complement sign rules for all four input sign combinations.

TABLE II
Custom Arithmetic Functions

Function	Algorithm	Complexity
`my_mul(a,b)`	Add `a` exactly `b` times	O(*b*)
`my_div(a,b)`	Subtract `b` until `a<b`	O(*a*/*b*)
`my_mod(a,b)`	Remainder after subtraction	O(*a*/*b*)
`my_clamp(v,lo,hi)`	Conditional min/max	O(1)

V. String Operations – string.c

All string manipulation is performed without including `stdio.h` or `string.h`. The key functions are:

- **`my_strlen`**: Pointer walk until `\0`, returning the byte distance from start.
- **`my_strcpy`**: Character-by-character copy with an explicit null-terminator written at the end.
- **`my_strcmp`**: Simultaneous walk of two strings; returns the difference of the first differing characters cast to `unsigned char` to handle extended ASCII correctly.
- **`my_int_to_str`**: Converts an integer to its decimal ASCII representation by repeatedly extracting the last digit via `my_mod(n, 10)`, converting it with `'0' + digit`, then calling `my_str_reverse` on the resulting buffer.

`my_int_to_str` is critical for score rendering because the game cannot use `printf` or `sprintf`.

VI. Terminal Rendering – screen.c

A. ANSI Escape Codes

All visual output is produced by writing ANSI/VT100 control sequences to `stdout` via the `write()` and `putchar()` system calls. Table III lists the primary sequences used.

TABLE III
ANSI Escape Sequences Used in Snake-OS

Sequence	Effect
`\033[2J`	Clear entire screen
`\033[y;xH`	Move cursor to row *y*, col *x*
`\033[?25l`	Hide cursor
`\033[?1049h`	Switch to alternate screen buffer
`\033[Nm`	Set foreground colour code *N*
`\033[0m`	Reset all attributes

B. Dynamic Board Resizing

Board dimensions are computed at runtime using the POSIX `ioctl(TIOCGWINSZ)` system call, which populates a `winsize` struct with the current terminal columns and rows. This call is issued on every frame, so the board automatically adapts when the user resizes the terminal window. Tail segments that fall outside the new boundaries are pruned from the linked list to prevent rendering artefacts.

C. Colour Scheme

The snake body is rendered with a gradient effect: the head uses bright green (`\033[92m`), the first third of the tail uses bold cyan, the middle third uses normal cyan, and the final third uses dim cyan. The head character changes to `^`, `v`, `<`, or `>` based on the current direction of travel.

VII. Keyboard Input – `keyboard.c`

A. Raw Mode Configuration

The terminal is placed in raw mode by retrieving the current `termios` struct via `tcgetattr()`, clearing the `ICANON` and `ECHO` flags in `c_lflag`, setting `VMIN = 0` and `VTIME = 0` in `c_cc`, and applying the new struct via `tcsetattr()`. The original settings are saved and restored at program exit via `atexit(keyboard_restore)`.

Setting `VMIN = 0` and `VTIME = 0` makes `read()` completely non-blocking: it returns immediately with 0 bytes if no key is pending, allowing the game loop to advance each frame without stalling on input.

B. Arrow Key Detection

Arrow keys transmit a three-byte escape sequence: `ESC` (0x1B), `[` (0x5B), and a final byte (`A` = Up, `B` = Down, `C` = Right, `D` = Left). The `map_key()` function detects an initial `ESC` byte, reads the next two bytes, and translates the sequence to the corresponding `W`/`A`/`S`/`D` character so the game logic operates on a single unified key set.

VIII. Game Engine – `snake.c`

A. Data Structures

The snake body is represented as a singly linked list of `Segment` nodes, each allocated from `VRAM` via `my_alloc()`:

```
typedef struct Segment {
    int x, y;
    struct Segment *next;
} Segment;
```

Obstacles are maintained as a linked list of `Obstacle` nodes, each carrying a `type` field (0 = static `#`, 1 = horizontally moving `M`), a direction, and boundary values.

B. Movement Mechanism

Snake movement is implemented as a constant-time head operation combined with a linear-time tail removal, executed once per frame:

1) `tail_push_front(old_x, old_y)` allocates a new `Segment` node and prepends it to the list in O(1).
2) The `Snake` head struct advances to the new position.
3) `tail_pop_back()` traverses the list to the penultimate node, erases the last segment from the screen, frees its memory via `my_dealloc()`, and nulls the previous pointer in O(n).

When food is consumed, `tail_pop_back()` is skipped for that frame, increasing the list length by one and visually growing the snake.

C. Reverse-Direction Prevention

Before accepting a new direction, the engine checks:

```
if (my_abs(new_dx + dir_x) == 0
    && my_abs(new_dy + dir_y) == 0)
    return; /* opposite direction -- ignore */
```

If the new and current direction vectors sum to zero in both components, the input is discarded, preventing the snake from reversing into itself.

D. Pseudo-Random Food Placement

Since `rand()` is prohibited, food coordinates are generated using the global tick counter `g_tick` with prime-number multipliers:

```
x = my_mod(my_mul(g_tick, 37) + 17, range_x)
    + PLAY_MIN_X;
y = my_mod(my_mul(g_tick, 53) + 11, range_y)
    + PLAY_MIN_Y;
```

The use of distinct prime multipliers (37 and 53) ensures the x and y coordinates follow independent pseudo-random sequences with long periods before repeating.

E. Game Modes

CLASSIC mode: Collision with any wall triggers an immediate game-over sequence including a flash death animation.

INFINITY mode: The snake wraps around to the opposite wall on boundary crossing, implemented by applying `my_mod` to the new coordinates against the play-area dimensions.

F. Food System

Three food types are supported, differentiated by point value and visual representation:

- **Normal** (`*`, $+1$ point): Always present; respawns immediately after consumption.
- **Bonus** (`+`, $+3$ points): Appears periodically; disappears after a timeout with a flicker animation.
- **Super** (`$`, $+5$ points): Rare; shorter timeout; flickers visibly before expiry.

G. Difficulty Progression

The game advances through four levels based on score thresholds. Each level adds static (`#`) and/or moving (`M`) obstacles. Moving obstacles traverse the board horizontally, reversing direction at assigned boundaries. Frame delay decreases with score:

$$d = \text{clamp}\Big(150000 - 10000 \cdot \Big\lfloor \frac{\text{score}}{50} \Big\rfloor,\ 60000,\ 150000\Big)\ \mu s$$

This yields approximately 6.7 FPS at score 0 and 16.7 FPS at maximum speed.

IX. Results and Evaluation

A. Feature Implementation Status

Table IV summarises the implemented features.

TABLE IV
FEATURE IMPLEMENTATION STATUS

Feature	Status
Custom memory allocator	Complete
No `*`, `/`, `%` operators	Complete
Raw terminal input	Complete
ANSI-only rendering	Complete
CLASSIC & INFINITY modes	Complete
Bonus food with timers	Complete
Dynamic terminal resize	Complete
Gradient snake rendering	Complete
Moving obstacles	Complete
Death animation	Complete
3-2-1 countdown	Complete
Score persistence (session)	Complete

B. Memory Allocator Performance

The 8 KB `VRAM` pool comfortably accommodates the game's peak allocation: a maximum-length snake of approximately 200 segments at `sizeof(Segment) = 12` bytes consumes 2,400 bytes. Combined with obstacle nodes and food structs, total peak usage remains below 4 KB, leaving over 50% of the pool free at all times.

C. Fragmentation Analysis

Internal fragmentation from `align_up()` padding is bounded at 7 bytes per allocation. External fragmentation is controlled by the coalescing algorithm; empirical testing showed no allocation failure under normal gameplay for sessions exceeding 1,000 frames.

D. Arithmetic Overhead

The loop-based arithmetic functions operate on small values – grid coordinates $\leq$ 200 and scores $\leq$ 500. The O(b) complexity of `my_mul` is negligible at these scales, and no perceptible latency was observed in the frame loop.

X. DISCUSSION

A. Limitations

The fixed 8 KB memory pool represents a hard ceiling on game complexity. Additionally, the pseudo-random generator based on `g_tick` is deterministic at fixed frame rates with no user interaction; a linear feedback shift register would improve unpredictability.

B. Portability

The project uses POSIX APIs (`termios`, `ioctl`, `read`, `write`, `usleep`) available on Linux and macOS. It is not directly portable to Windows without a compatibility layer.

C. Future Work

Potential extensions include: (1) a doubly-linked tail list to achieve O(1) `tail_pop_back`; (2) a linear feedback shift register for improved pseudo-randomness; (3) persistent high-score storage via a file descriptor; and (4) multiplayer support over a shared pseudo-terminal.

XI. CONCLUSION

Snake-OS demonstrates that a complete, visually polished, and interactive application can be built in C by re-implementing every layer ordinarily provided by the standard library. The project covers memory allocation and fragmentation management, loop-based arithmetic, raw terminal I/O, ANSI escape code rendering, linked list data structures, and pseudo-random sequence generation – all within a single cohesive application.

The result is both a functional game and a concrete illustration of systems programming concepts typically encountered only in abstract form. The deliberate constraints – no `malloc`, no hardware operators, no *ncurses* – force a deep understanding of what those abstractions actually do, making Snake-OS a valuable pedagogical exercise in bare-metal C programming.

REFERENCES

[1] Z. Ben-Halim, T. Dickey *et al.*, "ncurses – new curses," Free Software Foundation, 1993–2024. [Online]. Available: https://invisible-island.net/ncurses/

[2] D. Lea, "A memory allocator," 1996. [Online]. Available: http://gee.cs.oswego.edu/dl/html/malloc.html

[3] J. Evans, "A scalable concurrent `malloc(3)` implementation for FreeBSD," in *Proc. BSDCan*, 2006.

[4] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.

[5] J. B. Peatman, *Design with PIC Microcontrollers*. Prentice Hall, 1998.

[6] IEEE Std 1003.1-2017, "The Open Group Base Specifications Issue 7 – General Terminal Interface," IEEE/The Open Group, 2018.

[7] ECMA International, "Control Functions for Coded Character Sets," Standard ECMA-48, 5th ed., 1991.

[8] ISO/IEC 9899:2011, "Programming Languages – C," International Organization for Standardization, 2011.